



fred_data_fetcher

A Reusable Pipeline for Fetching Economic Data from the FRED API

Author: [eoneine](#) | Built on Cookiecutter Data Science (CCDS) Template



What is FRED?

Core Concept

FRED is a free public macroeconomic database maintained by the St. Louis Fed.

Database Scale

800,000+

Economic data series available from national and international sources.

Key Indicators

GDP & Economic Growth

Unemployment Rate (UNRATE)

CPI & Interest Rates

Money Supply (M1/M2)

Why This Matters: The Business Case

Market Barrier

Bloomberg/Refinitiv cost **\$22k-\$24k/year** — prohibitive for many.

The Solution

FRED provides free economic data; this project automates and cleans access for scalable analysis.

Target Audience

- Data Scientists & Investors
- Academic Students
- Emerging Fintechs

Core Impact: This project democratizes data access for the financial community.



Project Overview

Key Features

Built on the Cloud Community Data Solution (CCDS) architecture.

- API Ingestion
- Data Cleaning & Validation
- Reshaping (Long/Wide Formats)
- Automated Export (.csv, .json)
- CLI Interface for Pipeline Automation

```
# High-level pipeline call
```

```
from fred_fetcher import FREDClient
```

```
pipeline = FREDClient(  
    api_key="FRED_API_KEY",  
    series=["GDP", "UNRATE"]  
)
```

```
df = pipeline.run_ingestion()  
df.reshape(mode="long").export("data.csv")
```

How the Pipeline Works: End-to-End Workflow



Automated sequence from user input to structured data.



FRED API: Technical Specifications & Quirks

GET

/fred/series/observations

Request Parameters

`series_id`: The ID for a FRED data series (e.g., 'GDP', 'CPIAUCSL')

`api_key`: A 32-character alphanumeric FRED API key

`file_type`: Response format ('json' required for this project)

`observation_start`: Start date of the observation period (YYYY-MM-DD)

`observation_end`: End date of the observation period (YYYY-MM-DD)

⚠ API Quirks & Edge Cases

All data is returned as **string** typing (even numeric values). Missing data is marked with a period "." instead of null.



Data Cleaning & Sanitization

3-Step Sanitization Process

1. Imputation

Handle placeholders ('.' → NaN)

2. Type Casting

Convert to Datetime & Float64

3. Deduplication

Remove redundant observation rows

```
# Sanitization pipeline logic
def clean_data(df):
    # Step 1: Imputation
    df = df.replace('.', np.nan)
    # Step 2: Casting
    df['value'] = df['value'].astype(float)
    df['date'] = pd.to_datetime(df['date'])
    # Step 3: Deduplicate
    return df.drop_duplicates()
```

Flexible Output Format: Long vs. Wide

Formatting Control

The output structure is determined by the `--format` CLI flag, allowing seamless integration with various data workflows.

Long Format (Standard)

```
[date, series_id, value]
```

Good for Tidy Data

Optimized for database ingestion, BI tools, and programmatic analysis using Melt/Cast logic.

Wide Format (Pivoted)

```
[date, GDP, UNRATE, ... ]
```

Good for Spreadsheets

Optimized for human readability, Excel reporting, and quick side-by-side metric comparisons.



Command-Line Interface (CLI) Usage

CLI

`python run_pipeline.py`

```
$ python run_pipeline.py # 1. Default run
$ python run_pipeline.py --series GDP FEDFUNDS # 2. Specify series
$ python run_pipeline.py --start 2020-01-01 # 3. Custom start date
$ python run_pipeline.py --format wide # 4. Change data format

[OUTPUT] Generated file:
dataset_fred_GDP_UNRATE_2020-01-01_to_2024-12-31_long.csv
```



Project Directory Structure

Maintainability & CCDS

The project follows the **Cookiecutter Data Science (CCDS)** standard to ensure logical separation between production code, experimental research, and processed assets.

Key Benefits:

- Clean separation of concerns.
- Predictable environment for CI/CD.
- Optimized for data team collaboration.

Project root structure

```
├── data/
│   └── processed/
├── fred_data_pipeline/
├── notebooks/
├── tests/
└── run_pipeline.py
```



Architecture: Object-Oriented & Modular Design

Hybrid Paradigm Architecture

FREDClient (OO): Encapsulates API state, authentication, and endpoint orchestration.

Logic (Functional): Pure functions handle data cleaning and transformation for testability.

Modular Layers:

1. Config Layer
2. Ingestion (API)
3. Processing (Cleaning)
4. Orchestration (Pipeline)

```
# client.py

class FREDClient:
    def __init__(self, api_key):
        self.base_url = "..."
        self.api_key = api_key

    def fetch_series(self, sid):
        """Handles API request"""
        endpoint = f"{self.base_url}/..."
        return requests.get(endpoint)
```



Unit Testing with Pytest

Core Validation Suite

- ✓ `test_clean_data_converts_types`
- ✓ `test_reshape_data_wide`
- ✓ `test_reshape_rejects_unknown`

The test suite ensures data integrity during transformations and validates error handling for invalid FRED API responses.

```
# test_cleaner.py
import pytest
from src.cleaner import clean_data

def
test_clean_data_converts_types(sample_df):
    """Ensure columns are datetime objects."""
    cleaned = clean_data(sample_df)
    assert cleaned['date'].dtype ==
        'datetime64'
    assert cleaned['value'].notnull().all()
```



Future Improvements & Next Steps

Future Iterations Roadmap

Planned enhancements for future iterations focusing on library expansion and automation.

Expanded Scope

Expanded endpoints (search/meta)

Broader library ecosystem coverage

Integration & Automation

Advanced data visualizations

Statistical modeling integration

Automation via Apache Airflow